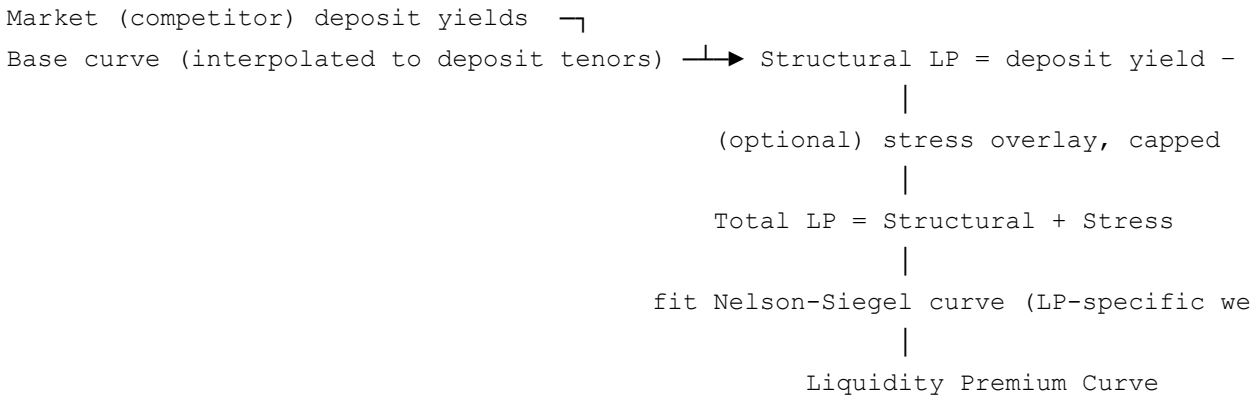
# How We Build the Liquidity Premium Curve — Code & Data Walkthrough

Where the base curve ( `docs/base_curve_construction.md` ) prices pure term risk, we use the liquidity premium (LP) curve to price the extra spread a funding source demands for being less liquid than a traded instrument of the same tenor. Combined, `Base Rate + LP = Full FTP Curve` . This lives in the same module, `curve_model/ftp_curve_model.py` .

## How we build it




We deliberately don't feed our own quoted deposit rate into this model at all — every deposit input, here and in `docs/base_curve_construction.md` , is a competitor/market observation. See "What this means for the LP number" below for why that's a real interpretive choice, not just a data-availability workaround.

## Step 1 — Structural LP ( `compute_structural_lp` )

We start with the clean economic liquidity spread: a market deposit yield at a tenor, minus the base curve's yield at that same tenor (interpolated via PCHIP onto the deposit tenors). No stress adjustment here — this is the observed, as-is spread. We apply two safeguards by default: winsorization (clipping the top ~2% of observations so a single outlier can't dominate the fit) and a floor at zero (a deposit trading *inside* the base curve doesn't get treated as a negative liquidity cost).

## Step 2 — Stress overlay ( `compute_stress_lp` ) — optional, capped

We scale the structural LP up at longer tenors with a multiplicative stress factor, `1 + α·(1 − e^(−t/τ))` — stress builds toward a horizon `τ`, capped at a maximum proportional uplift `α`. We keep only the **incremental** stress contribution as its own column, rather than blending it into structural LP, and we cap that increment in basis points too, so a single long-tenor outlier can't run away with the curve. This is a scenario layer we add on top, not part of our "base case" structural spread.

## Step 3 — Total LP and the fit target

`Total_LP = Structural_LP + Stress_LP` (Stress_LP sits at all-zero when we've disabled the overlay). Our diagnostics table carries both components separately alongside the total:

| Tenor | Structural LP | Stress LP | Total LP |
|---|---|---|---|
| 0.25 | 0.42% | 0.00% | 0.42% |
| 2.00 | 0.68% | 0.14% | 0.82% |
| 10.00 | 1.05% | 0.31% | 1.36% |

**Which column we actually fit is a named, reviewable config choice** (`CurveConfig.lp_fit_target`), not a hardcoded assumption — we can set it to `Total_LP` (structural + stress) or `Stress_LP` (the overlay alone). We built it this way specifically because an earlier version of this model fitted the curve to the stress overlay only, which understates the curve's own genuine structural spread. We treat this as a decision worth making deliberately on every real run rather than trusting the default — see "Where we hit real limits" below.

## Step 4 — Fitting the LP curve

We fit the LP curve with its own Nelson-Siegel bounds, weights, and starting guess — deliberately different from the base curve's:

- **Weights** (`build_lp_weights`) over-weight the short end (2.5×) and belly (1.5×) relative to the long end (0.8×), so we stabilize the curve where LP data is typically thinnest and most volatile, and keep unstable long-end extrapolation from dominating the fit.
- **Bounds** (`build_lp_bounds`) constrain $\beta_0$ (long-term level) to a small, strictly non-negative range — we expect LP to be a modest, smooth, positive spread, not a curve with its own independent shape the way the base curve has.
- We also built an anchor-constraint helper (`build_lp_anchor_constraints`) to pin LP

near zero at very short tenor — it's available, but not currently wired into the fit call; we kept it as an option for a future revision.

We floor the resulting curve at zero ( `max(fitted LP, 0)` ) before adding it to the base curve.

## What this means for the LP number

Because every deposit input is a competitor/market observation, the curve this step produces is a **market-observed liquidity premium** — what less-liquid funding costs across the competitor set we track — not a direct measurement of our own institution's realised funding cost. That's a deliberate choice, not a limitation to work around: it means the LP curve, and the Full FTP Curve built on top of it ( `docs/full_ftp_rate.md` ), prices every account against an external, independently-observable market benchmark rather than our own book's particular funding mix on any given day. Worth being explicit about that framing anywhere this curve gets used or reported — "market-implied liquidity premium," not "our own cost of liquidity."

## Where we hit real limits

- **Data gets sparse, and it's worse without the base shift.** We fit LP off competitor deposit-rate data, which is often thinner and less granular than the T-bill/bond universe feeding the base curve — especially at the short end, where competitor rate cards may only publish a handful of tenors. With the 3-month base shift off by default (see `docs/base_curve_construction.md` ), there's no short-end anchor pulling the base curve toward observed short deposit rates before we compute the LP spread, which can leave the LP fit more sensitive to whatever few short-tenor points we have. Our stressed-parameter overlay (Step 2) exists partly to give us a documented, bounded way to widen the curve for scenario testing, rather than leaning on a thin point estimate as the single answer.
- **The fit-target choice genuinely matters.** As above — `lp_fit_target` changes the resulting curve meaningfully; we treat it as a modeling decision to document per run, not a default to leave unexamined.
- **Competitor data, few tenors, and out of our control.** Our clean-spread assumption (deposit yield – base yield) degrades if the competitor rate data we can actually observe doesn't span enough tenors to fit a smooth curve — the weighting scheme helps, but it can't manufacture data we don't have, and unlike our own book, we can't simply collect more of it on demand.

See also `docs/base_curve_construction.md` and

curve_model/CURVE_CONSTRUCTION_ENGINE.md .